

بسمه تعالی

درس سیستم های نهفته - نیمسال تحصیلی ۰۴-۰۵

دانشگاه صنعتی شریف - دانشکده مهندسی برق



گزارش بخش دوم

امیرحسین مطیعی - شماره دانشجویی ۴۰۱۱۰۲۵۵۳

چکیده

در بخش دوم تمرین، سامانه پایگاه داده توزیع شده پیاده سازی شده در بخش اول با اضافه کردن یک لایه Cache مبتنی بر Memcached توسعه داده شد. هدف اصلی از این تغییر، کاهش زمان پاسخ گویی به درخواست های تکراری، کاهش تعداد دسترسی ها به پایگاه داده SQLite و جلوگیری از ارسال مجدد درخواست های شبکه ای غیر ضروری به نودهای Slave بود. معماری سیستم همچنان شامل یک نود Master و دو نود Slave است که هر کدام روی یک ماشین مجازی مستقل با سیستم عامل Ubuntu 22.04 اجرا می شوند. هر نود علاوه بر پایگاه داده محلی SQLite، دارای یک نمونه مستقل از سرویس Memcached است که در حافظه RAM اجرا می شود. اپراتور درخواست های خود را تنها برای Master ارسال می کند. Master در ابتدای پردازش هر درخواست، Cache محلی خود را بررسی می کند. اگر داده در Cache موجود باشد، پاسخ مستقیماً از حافظه بازگردانده می شود. اگر داده در Cache وجود نداشته باشد، Master ابتدا SQLite محلی خود را جستجو می کند و در صورت پیدا نشدن داده، درخواست را به ترتیب برای Slave1 و Slave2 ارسال می کند. پس از پیدا شدن داده در هر یک از نودها، نتیجه در Cache نود مربوطه و همچنین در Cache اصلی Master ذخیره می شود تا درخواست های بعدی بدون مراجعه مجدد به SQLite یا نودهای دیگر پاسخ داده شوند. برای بررسی عملکرد این ساختار، همه سنسورهای موجود در سیستم در دو دور خوانده شدند. در دور اول، Cache اصلی Master خالی بود و داده ها از SQLite محلی یا از طریق ارتباط با Slave ها بازیابی شدند. در دور دوم، همان درخواست ها مجدداً ارسال شدند و همه پاسخ ها از Cache اصلی Master دریافت شدند. نتایج نشان داد که میانگین زمان پاسخ از حدود ۳۵۲۱ میکروثانیه در دور اول به حدود ۱۳۲ میکروثانیه در دور دوم کاهش یافته است. بنابراین، استفاده از Memcached در این سیستم موجب کاهش حدود ۹۶،۲۵ درصدی میانگین زمان پاسخ شد.

هدف بخش دوم

در سیستم بخش اول، هر درخواست برای یک سنسور مستلزم اجرای Query روی SQLite بود. همچنین اگر سنسور در Master قرار نداشت، Master باید درخواست را از طریق شبکه برای Slave1 و در صورت نیاز برای Slave2 ارسال می کرد. این روش برای اولین درخواست ضروری است، اما تکرار همان عملیات برای درخواست هایی که چندین بار ارسال می شوند، باعث افزایش زمان پاسخ، افزایش تعداد خواندن ها از دیسک و افزایش ترافیک شبکه می شود.

در بخش دوم، یک ساختار دولایه برای نگهداری داده ها در نظر گرفته شد. لایه اصلی و پایدار از SQLite تشکیل شده است و اطلاعات سنسورها و آخرین Reading های ثبت شده را نگهداری می کند. لایه دوم، Cache مبتنی بر Memcached است که داده های پرکاربرد یا اخیراً خوانده شده را به صورت موقت در حافظه RAM نگهداری می کند. به دلیل آنکه دسترسی به حافظه RAM سریع تر از اجرای Query روی پایگاه داده و سریع تر از ارسال درخواست شبکه ای است، انتظار می رود درخواست های تکراری با زمان بسیار کمتری پاسخ داده شوند.

ساختار خواندن و نوشتن داده در SQLite و Memcached

در طراحی سیستم دو لایه پایگاه داده، برای هر نود یک ساختار ترکیبی شامل پایگاه داده اصلی SQLite و حافظه کش Memcached در نظر گرفته شده است. به عنوان لایه دائمی ذخیره سازی داده ها عمل می کند و تمامی اطلاعات سنسورها و مقادیر ثبت شده در آن نگهداری می شوند. در مقابل، Memcached به عنوان یک لایه موقت با سرعت دسترسی بالا در حافظه RAM استفاده شده است تا درخواست های پرتکرار بدون نیاز به مراجعه مداوم به پایگاه داده پاسخ داده شوند.

فرآیند دریافت اطلاعات سنسور به صورت سلسله مراتبی انجام می شود. هنگامی که یک درخواست برای دریافت مقدار آخرین ثبت شده یک سنسور ارسال می شود، ابتدا کلید مربوط به آن سنسور بر اساس نوع و شناسه سنسور ساخته شده و در Memcached جستجو می شود. کلیدهای کش در این پروژه به صورت ترکیبی از نوع سنسور و شناسه آن ایجاد می شوند تا هر سنسور دارای یک مقدار مستقل در حافظه کش باشد.

در صورتی که داده مورد نظر در Memcached موجود باشد، مقدار ذخیره شده مستقیماً از حافظه خوانده شده و بدون مراجعه به SQLite به درخواست کننده بازگردانده می شود. این حالت باعث کاهش زمان پاسخ دهی و کاهش تعداد عملیات خواندن روی پایگاه داده اصلی می شود.

اگر داده در Memcached وجود نداشته باشد، سیستم این حالت را Cache Miss در نظر گرفته و درخواست را به SQLite ارسال می کند. در این مرحله، جدول های مربوط به اطلاعات سنسورها و داده های ثبت شده بررسی شده و آخرین مقدار ثبت شده برای سنسور مورد نظر استخراج می شود. پس از دریافت موفق داده از SQLite، نتیجه علاوه بر ارسال برای کاربر، در Memcached نیز ذخیره می شود تا درخواست های بعدی بتوانند مستقیماً از کش پاسخ داده شوند.

فرآیند نوشتن در Memcached تنها پس از یک خواندن موفق از SQLite انجام می شود. به عبارت دیگر، SQLite منبع اصلی و معتبر داده ها است و Memcached فقط یک نسخه موقت از آخرین داده خوانده شده را نگهداری می کند. در این پروژه عملیات نوشتن مستقیم داده های جدید سنسورها در Memcached انجام نمی شود، زیرا هدف کش افزایش سرعت خواندن اطلاعات موجود است و عملیات ثبت داده همچنان باید در پایگاه داده اصلی انجام شود.

در Master نیز همین ساختار دو لایه استفاده شده است. ابتدا کش محلی Master بررسی می شود. اگر داده در کش Master موجود باشد، پاسخ بدون ارتباط شبکه ای با Slave ها بازگردانده می شود. در غیر این صورت، Master ابتدا SQLite محلی خود را بررسی کرده و در صورت نبود داده، درخواست را به Slave ها ارسال می کند. زمانی که پاسخ از یکی از Slave ها دریافت شد، نتیجه در Cache Master ذخیره می شود تا درخواست های بعدی سریع تر پاسخ داده شوند.

به این ترتیب مسیر کلی دریافت داده در سیستم به شکل زیر است:

Client → Master Memcached → Master SQLite → Slave1 Memcached → Slave1 SQLite → Slave2 Memcached → Slave2 SQLite

البته در حالت معمول، با توجه به وجود کش، بیشتر درخواست های تکراری در همان مرحله اول پاسخ داده می شوند و نیاز به دسترسی به لایه های پایین تر ندارند.

معماری نهایی سیستم

سامانه از سه ماشین مجازی تشکیل شده است. نود Master با نشانی ۱۹۲.۱۶۸.۱۲۲.۲۲ روی پورت ۸۰۰۰ اجرا می شود. نود Slave1 با نشانی ۱۹۲.۱۶۸.۱۲۲.۱۸ روی پورت ۹۰۰۱ و نود Slave2 با نشانی ۱۹۲.۱۶۸.۱۲۲.۱۹۰ روی پورت ۹۰۰۲ اجرا می شوند. ارتباط مدیریتی با ماشین های مجازی از طریق SSH انجام شد، اما ارتباط برنامه ها با یکدیگر از طریق HTTP، نشانی IP و شماره Port برقرار می شود.

هر ماشین مجازی دارای یک سرویس مستقل Memcached روی نشانی ۱۲۷.۰.۰.۱ و پورت ۱۱۲۱۱ است. محدود شدن Memcached به رابط loopback موجب می شود سرویس Cache مستقیماً در شبکه در دسترس نباشد و تنها برنامه ای که روی همان ماشین مجازی اجرا شده است بتواند به آن متصل شود. این موضوع از نظر امنیتی نیز مناسب تر است، زیرا نیازی به در معرض شبکه قرار دادن پورت Memcached وجود ندارد.

ساختار کلی پردازش یک درخواست در Master به این صورت است:

```
Client
|
v
Master Memcached
|
|-- Cache Hit --> Return response
|
|-- Cache Miss
|
v
Master SQLite
|
|-- Found --> Store in Master cache --> Return response
|
|-- Not Found
|
v
Slave1
|
|-- Found --> Store in Master cache --> Return response
|
|-- Not Found
|
v
Slave2
|
|-- Found --> Store in Master cache --> Return response
|
|-- Not Found --> Return HTTP 404
```

هر Slave نیز ساختار مشابهی در سطح محلی دارد. هنگامی که Slave درخواستی از Master دریافت می کند، ابتدا Cache محلی خود را بررسی می کند. در صورت Cache Hit، داده مستقیماً از Memcached بازگردانده می شود. در صورت Cache Miss، پایگاه داده SQLite محلی جستجو می شود و اگر داده پیدا شود، پیش از ارسال پاسخ در Cache همان Slave ذخیره خواهد شد.

نصب و راه اندازی Memcached

برای پیاده سازی لایه Cache، بسته های memcached، libmemcached-dev و ابزارهای مورد نیاز روی هر سه ماشین مجازی نصب شدند. سرویس Memcached پس از نصب با systemd راه اندازی شد و در زمان Boot نیز قابل فعال شدن است. کتابخانه libmemcached-dev امکان برقراری ارتباط برنامه های C++ با سرویس Memcached را فراهم می کند.

دستورات اصلی نصب به شکل زیر بودند:

```
sudo apt update
```

```
sudo apt install memcached libmemcached-dev libmemcached-tools netcat-openbsd
```

سرویس Memcached با دستور زیر اجرا شد:

```
sudo systemctl start memcached
```

و وضعیت آن با دستور زیر قابل بررسی بود:

```
sudo systemctl status memcached
```

در آزمایش انجام شده، دستور بررسی پورت نشان داد که Memcached فقط روی ۱۲۷.۰.۰.۱:۱۱۲۱۱ در حال Listen است. به همین دلیل، پاک کردن Cache از سیستم Host با ارسال مستقیم دستور به IP ماشین مجازی امکان پذیر نبود و عملیات flush_all از داخل همان VM انجام شد.

فایل های پیکربندی

برای جلوگیری از قرار دادن ثابت IP، Port و مسیر پایگاه داده در کد، تمام تنظیمات از فایل های config خوانده می شوند. فایل پیکربندی Master شامل پورت سرویس HTTP، مسیر master.db، نشانی و پورت دو Slave، Timeout ارتباط با Slave ها، نشانی Memcached، پورت Memcached و مدت اعتبار داده های Cache است. در تنظیمات نهایی، مدت اعتبار یا TTL داده های Cache برابر ۳۰۰ ثانیه در نظر گرفته شد. در نتیجه، هر مقدار ذخیره شده در Cache حداکثر تا پنج دقیقه معتبر است و پس از آن Memcached اجازه دارد آن را منقضی کند. استفاده از TTL مانع باقی ماندن دائمی مقادیر قدیمی در حافظه می شود.

Slave ها نیز پورت HTTP، مسیر پایگاه داده محلی، نشانی و پورت Memcached و مقدار TTL را از فایل های پیکربندی خود دریافت می کنند. در نتیجه، انتقال برنامه به یک شبکه یا ماشین دیگر تنها با ویرایش فایل پیکربندی امکان پذیر است و نیازی به تغییر یا کامپایل مجدد کد منبع وجود ندارد.

کامپایل برنامه ها

برای Master، Slave1 و Slave2 فایل Makefile مستقل تهیه شد. در بخش دوم، فایل های cache.cpp و cache.h به هر برنامه اضافه شدند و در مرحله Link نیز کتابخانه libmemcached در کنار SQLite قرار گرفت. برنامه Master علاوه بر کد اصلی، شامل کد ارتباط HTTP با Slave ها، کد مدیریت Cache و کتابخانه Mongoose است. برنامه های Slave نیز شامل کد اصلی، کد Cache و فایل Mongoose هستند.

دستورهای کامپایل هر برنامه به شکل زیر اجرا شدند:

make clean

make

اجرای make clean فایل های Object و فایل اجرایی قبلی را حذف می کند و دستور make همه فایل ها را مجدداً کامپایل و Link می کند. کامپایل موفق هر سه برنامه نشان داد که وابستگی های SQLite، Memcached و Mongoose به درستی در Makefile ها قرار گرفته اند.

کامپایل موفق برنامه Master

```
amir@embedded-master:~/master$ make clean
rm -f master main.o http_client.o cache.o mongoose.o
amir@embedded-master:~/master$ make
g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic -I../mongoose -c main.cpp -o main.o
g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic -I../mongoose -c http_client.cpp -o http_client.o
g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic -I../mongoose -c -o cache.o cache.cpp
gcc -O2 -Wall -Wextra -I../mongoose -c ../mongoose/mongoose.c -o mongoose.o
g++ main.o http_client.o cache.o mongoose.o -lsqlite3 -lmemcached -o master
```

کامپایل موفق برنامه Slave1

```
amir@embedded-slave1:~/slave1$ make clean
rm -f slave1 main.o cache.o mongoose.o
amir@embedded-slave1:~/slave1$ make
g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic -I../mongoose -c main.cpp -o main.o
g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic -I../mongoose -c cache.cpp -o cache.o
gcc -O2 -Wall -Wextra -I../mongoose -c ../mongoose/mongoose.c -o mongoose.o
g++ main.o cache.o mongoose.o -lsqlite3 -lmemcached -o slave1
```

کامپایل موفق برنامه Slave2

```
amir@embedded-slave2:~/slave2$ make clean
rm -f slave2 main.o cache.o mongoose.o
amir@embedded-slave2:~/slave2$ make
g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic -I../mongoose -c main.cpp -o main.o
g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic -I../mongoose -c cache.cpp -o cache.o
gcc -O2 -Wall -Wextra -I../mongoose -c ../mongoose/mongoose.c -o mongoose.o
g++ main.o cache.o mongoose.o -lsqlite3 -lmemcached -o slave2
```

اجرای نودهای سیستم

برای اجرای صحیح سیستم، ابتدا سرویس Memcached روی هر سه ماشین مجازی فعال شد. سپس برنامه های Slave1 و Slave2 اجرا شدند و در نهایت برنامه Master راه اندازی شد. ترتیب اجرای Slave ها پیش از Master الزامی مطلق نیست، اما باعث می شود Master از ابتدای اجرا بتواند درخواست های توزیع شده را بدون خطای اتصال پردازش کند.

برنامه Slave1 با دستور زیر اجرا شد:

cd ~/slave1

./slave1 config

برنامه Slave2 نیز با دستور زیر اجرا شد:

cd ~/slave2

./slave2 config

و برنامه Master با دستور زیر اجرا شد:

```
cd ~/master
```

```
./master config
```

در خروجی راه اندازی Master، فعال بودن Cache، نشانی Memcached، TTL، پورت HTTP و نشانی هر دو Slave نمایش داده شد. در خروجی Slave ها نیز پورت، مسیر پایگاه داده و وضعیت فعال بودن Cache چاپ شد.

اجرای برنامه Master و نمایش فعال بودن Cache

```
amir@embedded-master:~/master$ ./master config
92b7ac 3 mongoose.c:7806:mg_mgr_init    MG_IO_SIZE: 16384, TLS: none
92b7ac 3 mongoose.c:7719:mg_listen      1 4 http://0.0.0.0:8000
Master running on port 8000, database: master.db, cache: enabled
Master Memcached: 127.0.0.1:11211, TTL: 300 seconds
Slave1: 192.168.122.18:9001
Slave2: 192.168.122.190:9002
```

اجرای برنامه Slave1 و نمایش فعال بودن Cache

```
amir@embedded-slave1:~/slave1$ ./slave1 config
91164d 3 mongoose.c:7806:mg_mgr_init    MG_IO_SIZE: 16384, TLS: none
91164e 3 mongoose.c:7719:mg_listen      1 4 http://0.0.0.0:9001
Slave running on port 9001, database: slave1.db, cache: enabled
```

اجرای برنامه Slave2 و نمایش فعال بودن Cache

```
amir@embedded-slave2:~/slave2$ ./slave2 config
907d2f 3 mongoose.c:7806:mg_mgr_init    MG_IO_SIZE: 16384, TLS: none
907d2f 3 mongoose.c:7719:mg_listen      1 4 http://0.0.0.0:9002
Slave running on port 9002, database: slave2.db, cache: enabled
```

نحوه مقداردهی اولیه اتصال Cache

هنگام شروع برنامه، تنظیمات مربوط به Memcached از فایل config خوانده می شوند. سپس برنامه با استفاده از تابع مقداردهی Cache به سرویس Memcached موجود روی ۱۲۷.۰.۰.۱:۱۱۲۱۱ متصل می شود. نتیجه این عملیات در یک متغیر وضعیت ذخیره می شود تا برنامه بداند Cache در دسترس است یا خیر.

اگر اتصال به Memcached موفق باشد، پیام cache: enabled در خروجی برنامه نمایش داده می شود. اگر اتصال موفق نباشد، خطا در Log ثبت می شود، اما برنامه به طور کامل متوقف نمی شود. در این حالت سیستم می تواند بدون Cache به کار خود ادامه دهد و درخواست ها را مستقیماً از SQLite یا Slave ها پاسخ دهد. این رفتار باعث می شود خرابی موقت Cache به از کارافتادن کامل سامانه منجر نشود.

نحوه ساختن کلید Cache

برای هر سنسور یک کلید یکتا از ترکیب نوع سنسور و شناسه سنسور ایجاد می شود. استفاده همزمان از این دو مقدار اهمیت دارد، زیرا تنها استفاده از شناسه ممکن است در صورت وجود انواع متفاوت یا ارسال نوع اشتباه، نتیجه نادرستی تولید کند.

ساختار مفهومی کلید Cache به صورت زیر است:

sensor:<sensor_type>:<sensor_id>

برای نمونه، کلید سنسور دمای شماره ۲۰۱ به شکل زیر ساخته می شود:

sensor:temperature:201

نوع سنسور پیش از تولید کلید پاکسازی و نرمال سازی می شود تا تفاوت های غیرضروری در قالب ورودی باعث ساخت کلیدهای مختلف برای یک سنسور واحد نشود.

کش چگونه مقداردهی می شود

در شروع اجرای برنامه، Cache از قبل با تمام داده های SQLite پر نمی شود. این پروژه از روش Lazy Loading یا مقداردهی هنگام نیاز استفاده می کند. در این روش تنها داده هایی وارد Cache می شوند که حداقل یک بار درخواست شده اند.

هنگامی که درخواستی به Master می رسد، ابتدا کلید مربوط به سنسور ساخته و در Memcached جستجو می شود. اگر کلید وجود نداشته باشد، عملیات Cache Miss رخ می دهد. Master سپس SQLite محلی را بررسی می کند. اگر آخرین Reading سنسور در Master پیدا شود، اطلاعات سنسور به JSON تبدیل و با همان کلید و TTL برابر ۳۰۰ ثانیه در Cache اصلی Master ذخیره می شود. سپس پاسخ با مقدار source برابر master برای اپراتور ارسال می شود.

اگر سنسور در SQLite نود Master وجود نداشته باشد، درخواست را برای Slave1 ارسال می کند. Slave1 ابتدا Cache خود را بررسی کرده و در صورت نبود داده، SQLite محلی خود را می خواند. اگر داده پیدا شود، آن را در Cache محلی خود ذخیره و برای Master ارسال می کند. Master نیز بخش داده پاسخ Slave1 را استخراج کرده و همان مقدار را در Cache اصلی خود ذخیره می کند. پاسخ نهایی در این حالت با source برابر slave1 برای اپراتور ارسال می شود.

اگر داده در Slave1 نیز پیدا نشود، همین فرایند برای Slave2 تکرار می شود. پس از دریافت پاسخ موفق از Slave2، داده در Cache Master قرار گرفته و پاسخ با source برابر slave2 بازگردانده می شود.

بنابراین، Cache اصلی Master می تواند داده هایی را نگهداری کند که منبع اولیه آن ها Master، Slave1 یا Slave2 بوده است. این تصمیم باعث می شود درخواست تکراری اپراتور بدون آنکه Master دوباره SQLite یا شبکه را بررسی کند، مستقیماً از حافظه پاسخ داده شود.

چه زمانی داده از Cache خوانده می شود

داده زمانی از Cache خوانده می شود که برای ترکیب نوع و شناسه سنسور، یک کلید معتبر و منقضی نشده در Memcached وجود داشته باشد. این وضعیت معمولاً پس از آن رخ می دهد که همان سنسور حداقل یک بار با موفقیت از SQLite یا یکی از Slave ها خوانده شده باشد. در هر درخواست، بررسی Cache پیش از دسترسی به SQLite انجام می شود. اگر عملیات cache_get وضعیت Hit برگرداند، Master محتوای JSON ذخیره شده را دریافت می کند، مقدار source را برابر cache قرار می دهد، زمان سپری شده

را محاسبه می کند و پاسخ را فوراً برای Client ارسال می کند. پس از Cache Hit هیچ Query روی SQLite اجرا نمی شود و هیچ درخواست HTTP برای Slave1 یا Slave2 فرستاده نمی شود.

این موضوع در آزمایش های دستی نیز مشاهده شد. برای نمونه، اولین درخواست سنسور ۱۰۱ از SQLite نود Master پاسخ داده شد، اما درخواست دوم همان سنسور با source برابر cache بازگشت. همین رفتار برای سنسورهای متعلق به Slave1 و Slave2 نیز مشاهده شد.

اولین درخواست سنسور محلی Master و پاسخ با source برابر master

```
amir@embedded-master:~$ curl -s \
"http://127.0.0.1:8000/api/sensor?type=temperature&id=101" \
| python3 -m json.tool
{
  "source": "master",
  "response_time_us": 702,
  "data": {
    "sensor_id": "101",
    "sensor_type": "temperature",
    "sensor_name": "Floor1_Room101_Temp",
    "location": "Floor1_Room101",
    "value": "24.8",
    "unit": "C",
    "recorded_at": "2026-06-01 10:15:00"
  }
}
```

درخواست تکراری سنسور محلی Master و پاسخ با source برابر cache

```
amir@embedded-master:~$ curl -s \
"http://127.0.0.1:8000/api/sensor?type=temperature&id=101" \
| python3 -m json.tool
{
  "source": "cache",
  "response_time_us": 100,
  "data": {
    "sensor_id": "101",
    "sensor_type": "temperature",
    "sensor_name": "Floor1_Room101_Temp",
    "location": "Floor1_Room101",
    "value": "24.8",
    "unit": "C",
    "recorded_at": "2026-06-01 10:15:00"
  }
}
```

اولین درخواست سنسور متعلق به Slave1 و پاسخ با source برابر slave1

```
amir@embedded-master:~$ echo "flush_all" | nc localhost 11211
OK
^C
amir@embedded-master:~$ curl -s \
"http://127.0.0.1:8000/api/sensor?type=temperature&id=201" \
| python3 -m json.tool
{
  "source": "slave1",
  "response_time_us": 2496,
  "data": {
    "sensor_id": "201",
    "sensor_type": "temperature",
    "sensor_name": "Floor2_Room201_Temp",
    "location": "Floor2_Room201",
    "value": "25.3",
    "unit": "C",
    "recorded_at": "2026-06-01 10:15:00"
  }
}
```

درخواست تکراری سنسور متعلق به Slave1 و پاسخ با source برابر cache

```
amir@embedded-master:~$ curl -s \
"http://127.0.0.1:8000/api/sensor?type=temperature&id=201" \
| python3 -m json.tool
{
  "source": "cache",
  "response_time_us": 63,
  "data": {
    "sensor_id": "201",
    "sensor_type": "temperature",
    "sensor_name": "Floor2_Room201_Temp",
    "location": "Floor2_Room201",
    "value": "25.3",
    "unit": "C",
    "recorded_at": "2026-06-01 10:15:00"
  }
}
```

اولین درخواست سنسور متعلق به Slave2 و پاسخ با source برابر slave2

```
amir@embedded-master:~$ echo "flush_all" | nc localhost 11211
OK
^C
amir@embedded-master:~$ curl -s \
"http://127.0.0.1:8000/api/sensor?type=temperature&id=301" \
| python3 -m json.tool
{
  "source": "slave2",
  "response_time_us": 3535,
  "data": {
    "sensor_id": "301",
    "sensor_type": "temperature",
    "sensor_name": "Floor3_Room301_Temp",
    "location": "Floor3_Room301",
    "value": "23.1",
    "unit": "C",
    "recorded_at": "2026-06-01 10:15:00"
  }
}
```

درخواست تکراری سنسور متعلق به Slave2 و پاسخ با source برابر cache

```
amir@embedded-master:~$ curl -s "http://127.0.0.1:8000/api/sensor?type=temperature&id=301" | python3 -
m json.tool
{
  "source": "cache",
  "response_time_us": 76,
  "data": {
    "sensor_id": "301",
    "sensor_type": "temperature",
    "sensor_name": "Floor3_Room301_Temp",
    "location": "Floor3_Room301",
    "value": "23.1",
    "unit": "C",
    "recorded_at": "2026-06-01 10:15:00"
  }
}
```

چه زمانی داده از SQLite خوانده می شود

داده از SQLite زمانی خوانده می شود که کلید سنسور در Cache وجود نداشته باشد، منقضی شده باشد یا خواندن Cache با خطا مواجه شود. در این حالت، برنامه Query مربوط به آخرین Reading سنسور را روی SQLite محلی اجرا می کند.

برای انتخاب آخرین مقدار سنسور، داده ها بر اساس زمان ثبت به صورت نزولی مرتب می شوند و در صورت برابر بودن زمان، شناسه رکورد نیز به صورت نزولی بررسی می شود. اولین رکورد حاصل به عنوان جدیدترین Reading انتخاب می شود.

در Master، اگر SQLite محلی داده را در اختیار نداشته باشد، برنامه به Slave1 و سپس Slave2 مراجعه می کند. در هر Slave نیز همین قاعده برقرار است: Cache ابتدا بررسی می شود و تنها در صورت Cache Miss، SQLite محلی خوانده می شود.

در مسیر عادی API این بخش، داده جدیدی در SQLite نوشته نمی شود. پایگاه داده ها پیش از اجرا و از طریق اسکریپت های مقداردهی اولیه و فایل های CSV ساخته و پر می شوند. عملیات نوشتن زمان اجرای درخواست ها مربوط به Memcached است؛ یعنی داده خوانده شده از SQLite به Cache منتقل می شود. تمرکز بخش دوم پروژه بر بهینه سازی مسیر خواندن بوده است.

ساختار پاسخ و اندازه گیری زمان

خروجی هر درخواست موفق به شکل JSON بازگردانده می شود و شامل سه بخش اصلی است. فیلد source محل تأمین داده را مشخص می کند، فیلد response_time_us زمان پردازش درخواست را بر حسب میکروثانیه نشان می دهد و فیلد data شامل اطلاعات سنسور و آخرین مقدار ثبت شده آن است.

نمونه پاسخ Cache Hit به شکل زیر است:

```
{
  "source": "cache",
  "response_time_us": 122,
  "data": {
    "sensor_id": "101",
    "sensor_type": "temperature",
    "sensor_name": "Floor1_Room101_Temp",
    "location": "Floor1_Room101",
    "value": "24.8",
    "unit": "C",
    "recorded_at": "2026-06-01 10:15:00"
  }
}
```

زمان پاسخ درون برنامه Master از ابتدای ورود درخواست تا آماده شدن پاسخ اندازه گیری می شود. بنابراین در درخواست های توزیع شده، زمان ارتباط Master با Slave ها نیز در این مقدار تأثیر دارد. این زمان، زمان پردازش مشاهده شده در سمت سرور است و الزاماً تمام سربار اجرای دستور curl در سیستم Host را شامل نمی شود.

استفاده و تست سیستم

آزمایش ها از سیستم Host انجام شدند. در زمان تست، سه ترمینال SSH برای مشاهده خروجی Master، Slave1 و Slave2 باز نگه داشته شدند. ابتدا برنامه هر دو Slave اجرا شد و سپس Master راه اندازی گردید. در یک ترمینال جداگانه روی Host، درخواست های HTTP و اسکریپت تست سرعت اجرا شدند.

فرمت عمومی ارسال درخواست به Master به شکل زیر است:

```
curl -s \
```

```
"http://192.168.122.22:8000/api/sensor?type=<sensor_type>&id=<sensor_id>"
```

برای نمونه، درخواست سنسور ۱۰۱ به شکل زیر ارسال شد:

```
curl -s \
```

```
"http://192.168.122.22:8000/api/sensor?type=temperature&id=101"
```

در تست دستی، اولین درخواست سنسور ۱۰۱ از Master پاسخ داده شد و درخواست دوم از Cache بازگشت. برای بررسی مسیر Slave1، سنسور ۲۰۱ از نوع temperature درخواست شد. اولین پاسخ دارای source برابر slave1 بود و پاسخ تکراری از cache دریافت شد. همین آزمایش برای سنسور ۳۰۱ در Slave2 انجام شد و نتیجه مشابهی به دست آمد.

آماده سازی Cache برای تست سرعت

برای آنکه دور اول آزمایش واقعاً با Cache Miss در Master آغاز شود، پیش از اجرای اسکریپت، Cache نود Master پاک شد. از آنجا که Memcached فقط روی ۱۲۷.۰.۰.۱ گوش می داد، دستور پاک سازی از داخل VM نود Master اجرا شد:

```
echo "flush_all" | nc -q 1 localhost 11211
```

پاسخ OK نشان داد که داده های قبلی Cache پاک شده اند. پس از پاک سازی، اولین درخواست هر سنسور نمی توانست از Cache اصلی Master پاسخ داده شود و Master مجبور بود SQLite محلی یا Slave ها را بررسی کند.

```
amir@embedded-master:~$ echo "flush_all" | nc localhost 11211
OK
^C
amir@embedded-master:~$
```

اسکریپت تست سرعت

اسکریپت cache_speed_test.sh برای خواندن همه سنسورها در دو دور تهیه شد. اسکریپت برای هر درخواست، شناسه سنسور، نوع سنسور، منبع پاسخ و زمان پاسخ را استخراج می کند. نتیجه هر درخواست در ترمینال چاپ و همزمان در فایل cache_speed_result.csv ذخیره می شود.

اسکرپت از مسیر اصلی بخش دوم با دستور زیر اجرا شد:

bash scripts/cache_speed_test.sh

در دور اول، سنسورهای ۱۰۱ تا ۱۰۴ از Master، سنسورهای ۲۰۱ تا ۲۰۴ از Slave1 و سنسورهای ۳۰۱ تا ۳۰۴ از Slave2 پاسخ داده شدند. در دور دوم، همه دوازده سنسور با source برابر cache بازگشتند.

اجرای اسکرپت تست سرعت و نمایش نتایج دور اول و دوم

```
amir-hossein-motiei@amir-hossein-motiei-GP66-Leopard-11UG: ~/embedded/embedded_hw04/02$ bash scripts/cache_speed_test.sh
=====
Round 1: Cache Miss Test
=====
Sensor 101 -> master (8357 us)
Sensor 102 -> master (486 us)
Sensor 103 -> master (457 us)
Sensor 104 -> master (603 us)
Sensor 201 -> slave1 (9588 us)
Sensor 202 -> slave1 (1512 us)
Sensor 203 -> slave1 (1789 us)
Sensor 204 -> slave1 (1755 us)
Sensor 301 -> slave2 (9954 us)
Sensor 302 -> slave2 (2348 us)
Sensor 303 -> slave2 (2814 us)
Sensor 304 -> slave2 (2590 us)
=====
Round 2: Cache Hit Test
=====
Sensor 101 -> cache (182 us)
Sensor 102 -> cache (90 us)
Sensor 103 -> cache (205 us)
Sensor 104 -> cache (159 us)
Sensor 201 -> cache (66 us)
Sensor 202 -> cache (120 us)
Sensor 203 -> cache (117 us)
Sensor 204 -> cache (139 us)
Sensor 301 -> cache (109 us)
Sensor 302 -> cache (108 us)
Sensor 303 -> cache (73 us)
Sensor 304 -> cache (217 us)
```

محتوای فایل cache_speed_result.csv

```
amir-hossein-motiei@amir-hossein-motiei-GP66-Leopard-11UG: ~/embedded/embedded_hw04/02$ cat cache_speed_result.csv
round,sensor_id,sensor_type,source,response_time_us
1,101,temperature,master,8357
1,102,humidity,master,486
1,103,motion,master,457
1,104,temperature,master,603
1,201,temperature,slave1,9588
1,202,humidity,slave1,1512
1,203,motion,slave1,1789
1,204,co2,slave1,1755
1,301,temperature,slave2,9954
1,302,humidity,slave2,2348
1,303,motion,slave2,2814
1,304,smoke,slave2,2590
2,101,temperature,cache,182
2,102,humidity,cache,90
2,103,motion,cache,205
2,104,temperature,cache,159
2,201,temperature,cache,66
2,202,humidity,cache,120
2,203,motion,cache,117
2,204,co2,cache,139
2,301,temperature,cache,109
2,302,humidity,cache,108
2,303,motion,cache,73
2,304,smoke,cache,217
amir-hossein-motiei@amir-hossein-motiei-GP66-Leopard-11UG: ~/embedded/embedded_hw04/02$
```

نتایج دور اول و دوم

نتایج ثبت شده در آزمایش نهایی به شرح زیر است:

Benchmark Results

The measured results were:

Sensor ID	Sensor Type	Round 1 Source	Round 1 Time (μs)	Round 2 Source	Round 2 Time (μs)
101	temperature	master	8357	cache	182
102	humidity	master	486	cache	90
103	motion	master	457	cache	205
104	temperature	master	603	cache	159
201	temperature	slave1	9588	cache	66
202	humidity	slave1	1512	cache	120
203	motion	slave1	1789	cache	117
204	co2	slave1	1755	cache	139
301	temperature	slave2	9954	cache	109
302	humidity	slave2	2348	cache	108
303	motion	slave2	2814	cache	73
304	smoke	slave2	2590	cache	217

مجموع زمان پاسخ دوازده درخواست در دور اول برابر ۴۲۲۵۳ میکروثانیه بود. مجموع زمان همان درخواستها در دور دوم به ۱۵۸۵ میکروثانیه کاهش یافت. میانگین زمان هر درخواست در دور اول تقریباً ۳۵۲۱،۰۸ میکروثانیه و در دور دوم تقریباً ۱۳۲،۰۸ میکروثانیه بود. بنابراین، میانگین پاسخ در حالت Cache Hit حدود ۲۶،۶۶ برابر سریع تر از دور اول بود و کاهش میانگین زمان پاسخ تقریباً ۹۶،۲۵ درصد به دست آمد.

تحلیل زمان پاسخ سنسورهای Master

برای سنسورهای موجود در Master، میانگین زمان پاسخ دور اول برابر ۲۴۷۵،۷۵ میکروثانیه بود. میانگین زمان همان سنسورها در دور دوم به ۱۵۹ میکروثانیه رسید. در نتیجه، میانگین زمان پاسخ سنسورهای Master حدود ۹۳،۵۸ درصد کاهش یافت و پاسخ Cache تقریباً ۱۵،۵۷ برابر سریع تر بود.

زمان ۸۳۵۷ میکروثانیه برای اولین درخواست سنسور ۱۰۱ به طور محسوسی از سه سنسور دیگر Master بیشتر بود. علت احتمالی این اختلاف، سرد بودن وضعیت اولیه برنامه، بارگذاری اولیه صفحات پایگاه داده در حافظه سیستم عامل، ایجاد اولین اتصالها و هزینه های یک باره اجرای نخستین درخواست است. پس از گرم شدن فرآیند و منابع سیستم، درخواستهای بعدی SQLite زمان کمتری داشتند.

تحلیل زمان پاسخ سنسورهای Slave1

میانگین زمان دور اول برای سنسورهای Slave1 برابر ۳۶۶۱ میکروثانیه بود. این مقدار در دور دوم به ۱۱۰۰۵ میکروثانیه کاهش یافت. بنابراین، استفاده از Cache برای این گروه حدود ۹۶،۹۸ درصد کاهش زمان ایجاد کرد و پاسخ دور دوم تقریباً ۳۳،۱۳ برابر سریع تر بود.

درخواست اول سنسور ۲۰۱ زمان ۹۵۸۸ میکروثانیه داشت. این درخواست علاوه بر پردازش در Master، شامل ارسال HTTP به Slave1، پردازش درخواست در Slave، بررسی Cache محلی، خواندن داده و ارسال پاسخ به Master بود. در دور دوم، تمام این مراحل حذف شدند و داده تنها در ۶۶ میکروثانیه از Cache اصلی Master بازگردانده شد.

تحلیل زمان پاسخ سنسورهای Slave2

میانگین زمان دور اول سنسورهای Slave2 برابر ۴۴۲۶۰۵ میکروثانیه بود. میانگین زمان دور دوم این گروه برابر ۱۲۶۰۷۵ میکروثانیه به دست آمد. در این حالت زمان پاسخ حدود ۹۷،۱۴ درصد کاهش یافت و Cache تقریباً ۳۴،۹۲ برابر سریع تر عمل کرد.

زمان دور اول Slave2 از سایر گروه ها بیشتر بود، زیرا برای سنسوری که در Master و Slave1 وجود ندارد، Master ابتدا SQLite محلی خود را بررسی می کند، سپس یک درخواست برای Slave1 ارسال می کند و پس از دریافت پاسخ Not Found، درخواست دیگری برای Slave2 می فرستد. بنابراین مسیر رسیدن به Slave2 طولانی تر است. در دور دوم، Cache اصلی Master از این جست و جوی زنجیره ای جلوگیری کرد.

برای نمونه، سنسور ۳۰۱ در دور اول با زمان ۹۹۵۴ میکروثانیه از Slave2 دریافت شد، اما همان سنسور در دور دوم تنها در ۱۰۹ میکروثانیه از Cache پاسخ داده شد. این نتیجه نشان می دهد که بیشترین مزیت Cache برای داده هایی مشاهده می شود که در نودهای دورتر قرار دارند.

مقایسه دو حالت خواندن

در حالت Cache Miss، برنامه باید چند مرحله را انجام دهد. این مراحل شامل ساخت Query، باز کردن یا استفاده از اتصال SQLite، اجرای Statement، خواندن رکورد، تبدیل داده به JSON و در صورت دور بودن داده، برقراری ارتباط HTTP با Slaveها است. پس از پیدا شدن داده نیز یک عملیات cache_set برای ذخیره نتیجه انجام می شود. بنابراین، دور اول علاوه بر عملیات خواندن اصلی، هزینه مقداردی Cache را نیز در بر دارد.

در حالت Cache Hit، تنها کلید سنسور ساخته شده و با یک درخواست ساده از Memcached خوانده می شود. داده از قبل در قالب JSON در حافظه RAM قرار دارد و نیازی به اجرای Query، دسترسی به فایل SQLite، تبدیل مجدد رکورد یا ارتباط شبکه ای با Slaveها نیست. به همین دلیل زمان دور دوم برای همه سنسورها کمتر از دور اول است.

اختلاف زمان برای درخواست های توزیع شده بیشتر از درخواست های محلی بود. این نتیجه منطقی است، زیرا Cache در درخواست محلی تنها دسترسی SQLite را حذف می کند، اما در درخواست های Slave علاوه بر SQLite، ارتباط شبکه ای میان Master و Slave را نیز حذف می کند.

اگر داده در دور دوم از Cache خوانده نشود

در آزمایش نهایی، هر دوازده درخواست دور دوم با source برابر cache پاسخ داده شدند و رفتار سیستم مطابق انتظار بود. با این حال، در شرایط دیگر ممکن است یک درخواست دور دوم از Cache پاسخ داده نشود.

یکی از دلایل ممکن، ناموفق بودن درخواست دور اول است. اگر در دور اول سنسور پیدا نشود یا پاسخ Slave با خطا مواجه شود، داده‌ای برای ذخیره در Cache وجود ندارد و درخواست دوم نیز Cache Hit نخواهد داشت.

دلیل دیگر می‌تواند اشتباه بودن نوع سنسور یا شناسه باشد. کلید Cache از هر دو مقدار ساخته می‌شود. برای مثال، ارسال CO2 برای سنسوری که نوع واقعی آن temperature است، یک کلید متفاوت تولید می‌کند و مقدار قبلی پیدا نمی‌شود. این موضوع در مرحله اولیه توسعه برای سنسورهای ۱۰۴ و ۳۰۴ مشاهده شد. نوع واقعی سنسور ۱۰۴ برابر temperature و نوع سنسور ۳۰۴ برابر smoke بود. پس از اصلاح نوع‌ها در اسکریپت، هر دو سنسور در دور دوم از Cache پاسخ داده شدند.

انقضای TTL نیز می‌تواند علت Cache Miss در دور دوم باشد. اگر فاصله زمانی بین دو درخواست از مقدار TTL بیشتر شود، Memcached داده را منقضی شده در نظر می‌گیرد و برنامه باید دوباره SQLite را بخواند.

راه‌اندازی مجدد سرویس Memcached، خاموش شدن ماشین مجازی یا اجرای دستور flush_all میان دو دور نیز تمام یا بخشی از Cache را پاک می‌کند. علاوه بر این، اگر حافظه اختصاص یافته به Memcached پر شود، سرویس ممکن است طبق سیاست حذف خود داده‌های قدیمی را Evict کند.

ناموفق بودن عملیات cache_set نیز امکان دیگری است. در این سیستم، اگر داده از SQLite با موفقیت خوانده شود ولی ذخیره آن در Cache با خطا مواجه شود، پاسخ اصلی همچنان برای کاربر فرستاده می‌شود تا خرابی Cache مانع کار سیستم نشود. در چنین شرایطی، درخواست دوم دوباره از SQLite یا Slave پاسخ داده خواهد شد.

در نهایت، تغییر در روش ساخت یا نرمال‌سازی کلید میان دو درخواست می‌تواند باعث شود کلید تولید شده در دور دوم با کلید ذخیره شده در دور اول یکسان نباشد. به همین دلیل، تولید کلید در یک تابع مشترک انجام شده است تا Master و Slave رفتار یکنواختی داشته باشند.

بررسی صحت عملکرد

نتایج تست دستی و اسکریپت Benchmark چند ویژگی اصلی سیستم را تأیید کردند. نخست آنکه Master پیش از هر منبع دیگری Cache محلی خود را بررسی می‌کند. دوم آنکه داده‌های محلی Master در Cache ذخیره می‌شوند. سوم آنکه نتیجه دریافت شده از هر دو Slave نیز در Cache اصلی Master قرار می‌گیرد. چهارم آنکه فیلد source به درستی مسیر پاسخ را مشخص می‌کند. پنجم آنکه زمان پاسخ به میکروثانیه در خروجی قرار دارد. در نهایت، همه درخواست‌های دور دوم از Cache پاسخ داده شدند و هیچ مراجعه مجددی به SQLite یا Slave برای این داده‌ها لازم نبود.

محدودیت‌ها و نکات طراحی

Memcached یک حافظه موقت است و جایگزین SQLite محسوب نمی‌شود. در صورت خاموش شدن سرویس یا ماشین مجازی، داده‌های Cache از بین می‌روند، اما اطلاعات اصلی در SQLite باقی می‌مانند. بنابراین SQLite منبع اصلی و پایدار داده و Memcached تنها لایه شتاب‌دهنده خواندن است.

از آنجا که API فعلی بر خواندن داده تمرکز دارد، مسئله به‌روزرسانی هم‌زمان SQLite و Cache در زمان ثبت Reading جدید در این بخش پیاده‌سازی نشده است. TTL پنج دقیقه‌ای مانع باقی ماندن نامحدود داده در Cache می‌شود، اما در یک سامانه واقعی که Reading‌ها به‌صورت پیوسته تغییر می‌کنند، بهتر است هنگام ثبت مقدار جدید، کلید Cache مربوط به همان سنسور حذف یا با مقدار جدید جایگزین شود.

همچنین، زمان های ثبت شده به سخت افزار سیستم، وضعیت ماشین های مجازی، بار پردازنده، وضعیت Page Cache سیستم عامل و تأخیر شبکه وابسته هستند. بنابراین اعداد به دست آمده برای مقایسه دو مسیر در همین محیط معتبرند و نباید به عنوان معیار مطلق برای همه سخت افزارها در نظر گرفته شوند.

در بخش دوم پروژه، یک لایه Cache مبتنی بر Memcached به هر سه نود سیستم پایگاه داده توزیع شده اضافه شد. در ساختار نهایی، هر نود ابتدا Cache محلی را بررسی می کند و تنها در صورت Cache Miss به SQLite مراجعه می کند. Master علاوه بر داده های محلی، پاسخ های دریافت شده از Slave1 و Slave2 را نیز در Cache خود ذخیره می کند. این طراحی باعث می شود درخواست تکراری اپراتور مستقیماً در نود Master پاسخ داده شود.

آزمایش همه دوازده سنسور در دو دور نشان داد که در دور اول داده ها از Master، Slave1 و Slave2 بازیابی شدند، در حالی که در دور دوم همه پاسخ ها از Cache دریافت شدند. میانگین زمان پاسخ از ۳۵۲۱۰۰۸ میکروثانیه به ۱۳۲۰۰۸ میکروثانیه کاهش یافت. این مقدار معادل کاهش ۹۶٫۲۵ درصدی زمان پاسخ و افزایش سرعت تقریبی ۲۶٫۶۶ برابری است.

نتایج نشان می دهند که استفاده از Memcached به ویژه برای سنسورهای قرار گرفته در Slave ها مؤثر است، زیرا علاوه بر حذف دسترسی مجدد به SQLite، ارتباط شبکه ای و جست و جوی ترتیبی نودها را نیز حذف می کند. در مجموع، پیاده سازی بخش دوم با موفقیت انجام شد و الزامات مربوط به ساختار دولایه، اولویت بررسی Cache، ثبت زمان پاسخ، اجرای تست دو دور و تحلیل نتایج برآورده شدند.